

Reporte Técnico y Estratégico

Estándar Arquitectónico para la Reutilización de Software en una Consultora

Elaborado por: Gonzalo Gutiérrez Castillo

Contexto: Evaluación de la mejor estrategia arquitectónica para cumplir el objetivo empresarial de reutilizar y mantener proyectos de forma rentable y sostenida, tanto para clientes del mismo rubro como de rubros distintos.

1. Resumen Ejecutivo

El equipo de desarrollo enfrenta una decisión que no es meramente técnica, sino estratégica: ¿bajo qué filosofía arquitectónica debe operar la consultora?

Ambas posturas parten de los mismos fundamentos acordados — Clean Architecture, CQRS y DDD — pero difieren radicalmente en cómo aplicarlos, siendo la segunda la postura defendida por el autor del presente informe:

- **Enfoque Específico (YAGNI):** Diseñar cada sistema resolviendo única y exclusivamente lo que el cliente actual necesita hoy. Prioriza la velocidad de la primera entrega evitando abstracciones no solicitadas explícitamente.
- **Modelo de Plataforma (Repositorio Base):** Construir y mantener un repositorio institucional estructurado bajo Clean Architecture, CQRS y DDD, que actúe como punto de partida estandarizado para todos los proyectos. Cada nuevo desarrollo parte de ese núcleo, lo extiende y lo adapta, pero no lo reinventa.

La elección entre ambas filosofías determina si la empresa opera como un **taller artesanal** (donde cada proyecto se construye desde cero o se clona manualmente) o como una **fábrica de software** (donde se consolida un activo intelectual reutilizable que acelera cada proyecto subsiguiente).

2. Contexto Empresarial: Por Qué Esta Decisión Es Crítica Ahora

La consultora no opera en un modelo de producto único. Recibe proyectos de múltiples rubros y clientes, y ha comenzado a recibir proyectos repetidos dentro de un mismo rubro. Esta realidad distingue a la consultora de una startup que explora un modelo de negocio incierto, y es el factor que define cuál filosofía es adecuada.

- **El conocimiento de dominio se repite.** Entidades como personas, contactos, roles, autenticación, auditoría o facturación aparecen en casi todos los proyectos independientemente del rubro.
- **Los patrones de arquitectura son siempre los mismos.** Si acordamos usar Clean Architecture, CQRS y DDD en todos los proyectos, no tiene sentido implementarlos desde cero cada vez.
- **El equipo crece y rota.** La ausencia de un estándar institucional convierte cada proyecto en una "caja negra" que solo entiende quien lo construyó.

Estos tres factores hacen que el enfoque YAGNI, válido en contextos de incertidumbre total, sea un riesgo operativo en el contexto actual de la consultora.

3. Fundamentos Teóricos

La postura del Modelo de Plataforma no es una preferencia personal ni una apuesta al over-engineering. Está respaldada por principios consolidados de la ingeniería de software:

DRY (Don't Repeat Yourself) y SSOT (Single Source of Truth)

Formulado por Andy Hunt y Dave Thomas en *The Pragmatic Programmer* (1999). Postula que cada pieza de conocimiento debe tener una única representación en el sistema. El enfoque específico viola DRY al reinventar la lógica común en cada proyecto. El Modelo de Plataforma respeta SSOT al centralizar ese conocimiento en el repositorio base.

Principio Open/Closed (SOLID)

Acuñado por Bertrand Meyer y popularizado por Robert C. Martin. Un sistema debe estar abierto para su extensión pero cerrado para su modificación arbitraria. El repositorio base permite extender el sistema para cada nuevo cliente mediante módulos propios, sin alterar el núcleo probado. Esto no significa que el núcleo sea inmutable para siempre, sino que no se altera para satisfacer la particularidad de un cliente individual.

Software Product Line Engineering (SPLE)

Paradigma estandarizado por el Software Engineering Institute (SEI) de Carnegie Mellon. Define cómo construir un portafolio de productos a partir de un núcleo común. La consultora, al tener proyectos de múltiples clientes con patrones comunes, opera exactamente en ese escenario.

YAGNI y Deuda Técnica

Principio proveniente de Extreme Programming (XP). Es válido para equipos que genuinamente no saben qué necesitarán. Sin embargo, como advierte Martin Fowler, aplicar YAGNI ignorando la arquitectura base en contextos donde ya se sabe que el software se va a reutilizar genera niveles crecientes de deuda técnica que a largo plazo paralizan al equipo.

4. La Propuesta Concreta: Repositorio Base Institucional

La propuesta no consiste en construir un ERP, un SaaS o una plataforma de escala mundial. Consiste en mantener un repositorio institucional con la siguiente naturaleza:

- Estructurado bajo Clean Architecture, CQRS y DDD — los mismos patrones ya acordados por el equipo.
- Contiene las capas base (Domain, Application, Infrastructure, API) ya configuradas y listas.
- Define entidades de dominio comunes (como Persona, autenticación, auditoría) que se repiten en la mayoría de proyectos.
-

Por qué este modelo es superior al paquete NuGet tradicional

Un paquete NuGet rígido obliga a todos los proyectos a usar exactamente la misma versión del código base. El modelo shadcn resuelve esto:

- No hay acoplamiento de versiones entre proyectos.
- Cada proyecto toma lo que necesita y lo moldea libremente.
- Las modificaciones en un proyecto no afectan a los demás.

La disciplina del equipo define que **no se modifica el repositorio base a la ligera**. Los cambios al núcleo deben justificarse como mejoras globales, no como adaptaciones de un cliente particular.

5. Análisis Operativo y Financiero

Factor	Enfoque Específico (YAGNI)	Modelo de Plataforma (Repositorio Base)	Impacto para la Empresa
Costo del 1er Proyecto	Menor inversión inicial	Mayor tiempo de diseño base	El enfoque YAGNI es más barato a cortísimo plazo
Costo del 2do Proyecto	Alto (requiere limpiar y adaptar el clon)	Bajo (se parte del repositorio base probado)	El Modelo de Plataforma maximiza la rentabilidad al reutilizar
Margen de utilidad	Lineal, depende de horas vendidas	Exponencial, se cobra por solución completa invirtiendo menos horas	El Modelo de Plataforma genera mayor margen neto
Dependencia del talento	Alta (conocimiento en la cabeza del programador)	Baja (conocimiento en la arquitectura institucional)	Mayor seguridad operativa con el Modelo de Plataforma
Mantenimiento a largo plazo	Paralelo y costoso por proyecto	Centralizado y escalable	El modelo específico colapsa con el volumen

6. El Escenario Crítico: Reutilización Entre Proyectos

Opción A: Reutilización por Clonado (Enfoque Específico)

Cuando llega un segundo cliente del mismo rubro, el equipo clona el proyecto del primero y comienza a "limpiar" lo que era específico del cliente anterior. Este proceso consume horas no facturables y genera un riesgo adicional: **el código se bifurca**. Si se detecta un fallo de seguridad en el proyecto del Cliente 1, ese arreglo no llega automáticamente al Cliente 2. Un desarrollador deberá entrar manualmente a cada proyecto a replicar el parche, multiplicando el costo de mantenimiento con cada cliente nuevo.

Opción B: Reutilización por Extensión (Modelo de Plataforma)

Cuando llega un nuevo cliente, el equipo parte del repositorio base institucional, ya probado y estructurado. Las particularidades del cliente se implementan como extensiones propias del proyecto. Una corrección aplicada al repositorio base puede propagarse de forma controlada a los proyectos que corresponda. El equipo invierte su tiempo en crear nuevas funcionalidades, no en apagar incendios repetidos.

7. Continuidad Operativa: El Riesgo del "Bus Factor"

Uno de los mayores riesgos en consultoría de software es la rotación de personal.

Con el Enfoque Específico: Si un ingeniero que desarrolló cinco proyectos "a medida" abandona la empresa, esos sistemas se convierten en cajas negras. El nuevo programador deberá invertir semanas en descifrar la lógica particular que su predecesor inventó para cada cliente. Esto paraliza el soporte técnico, encarece el mantenimiento y daña la reputación ante el cliente final.

Con el Modelo de Plataforma: Si el ingeniero se va, el impacto es mínimo. El nuevo desarrollador solo necesita ser capacitado en el repositorio base institucional. Una vez que entiende la estructura estándar, comprende automáticamente cómo están contruidos todos los proyectos que parten de esa base, garantizando continuidad operativa sin interrupciones.

8. El Mito del "Software a Medida" como Activo Intelectual

Existe la percepción de que los proyectos anteriores son activos listos para ser revendidos. Esto es una ilusión conocida como el **síndrome del traje a medida**.

El software del "Hotel A" está acoplado a las particularidades de ese hotel. Venderlo al "Hotel B" implica que los desarrolladores inviertan cientos de horas en desarmarlo y adaptarlo — trabajo que no se puede facturar en su totalidad y que degrada la calidad del resultado.

El verdadero activo intelectual se genera **aislando lo universal en el repositorio base y manteniendo lo específico de cada cliente fuera de ese núcleo**. Eso sí puede reutilizarse, mejorarse y venderse como capacidad instalada.

9. Análisis de Objeciones Técnicas

Durante el debate han surgido objeciones técnicas concretas que vale la pena analizar:

Objeción: "Las abstracciones generan campos nullable innecesarios"

Esta objeción aplica a un paquete NuGet rígido que no se puede tocar. En el modelo propuesto, el desarrollador copia el código y lo adapta al proyecto. Si una entidad tiene campos que no aplican, simplemente se ajusta en ese proyecto. No hay caja negra que imponga campos inútiles.

Objeción: "Si Docente extiende Persona del núcleo y necesita ApellidoPaterno y ApellidoMaterno, el campo Surname de Persona queda sin uso"

Nuevamente, esto es un problema de un paquete inmutable. En el modelo shadcn, el desarrollador ajusta Persona en ese proyecto para reflejar exactamente lo que el dominio requiere. La disciplina no es "nunca toques el código copiado" sino "no modifiques el repositorio base por razones de un solo cliente".

En síntesis: las dos objeciones describen problemas de un paquete rígido, no del modelo propuesto.

10. Análisis de Riesgos del Modelo de Plataforma y su Mitigación

Riesgo	Plan de Mitigación
Tiempos de entrega iniciales más largos	El repositorio base funciona como boilerplate institucional: los proyectos arrancan con la arquitectura ya lista
Curva de aprendizaje para el equipo	Documentación interna clara del repositorio base y sus convenciones
Pérdida de flexibilidad ante datos muy atípicos	Uso de campos dinámicos (JSONB) o tablas de metadatos para particularidades que no ameritan modificar el modelo central
Riesgo de over-engineering	El núcleo se construye sobre patrones ya probados en proyectos reales de la consultora, no sobre suposiciones

11. Por Qué las Objeciones Filosóficas No Aplican a Este Contexto

"No estamos haciendo un SaaS"

Esta objeción confunde el modelo de ventas con el modelo de ingeniería. No se venden suscripciones, pero el objetivo de la gerencia es operar con una base común para N clientes. Sin una arquitectura centralizada, mantener diez proyectos del mismo rubro tiene el mismo costo que mantener diez proyectos totalmente distintos. Para que la reutilización sea rentable, la ingeniería interna debe comportarse como la de una plataforma, independientemente de cómo se facture al cliente.

"No estamos intentando construir otro Odoo"

Nadie propone construir la complejidad de un ERP global. Adoptar los patrones de diseño eficientes de Odoo — como una entidad central de Persona — no significa replicar su escala. Significa aplicar sus principios de escalabilidad al nicho específico de la consultora. Rechazar buenas prácticas de arquitectura bajo la excusa del tamaño actual es garantizar que el software colapse cuando el volumen de proyectos crezca.

"Las abstracciones deben surgir naturalmente para evitar over-engineering"

Este argumento es válido para equipos que genuinamente no conocen su dominio. La consultora ya ha trabajado con múltiples clientes y conoce perfectamente los patrones que se repiten. No se está adivinando ni especulando: se está sistematizando lo que ya se sabe que funciona. Esperar a que la abstracción "surja naturalmente" cuando la consultora está creciendo en volumen de proyectos es un lujo reactivo que generará una avalancha de refactorizaciones costosas.

12. Conclusión y Recomendación

Si la consultora recibiera proyectos aislados, sin intención de reutilización y sin crecimiento proyectado, el enfoque específico (YAGNI) sería perfectamente adecuado.

Sin embargo, la realidad de la consultora es otra: proyectos en múltiples rubros, clientes repetidos en el mismo rubro, un equipo que crece y rota, y un objetivo estratégico explícito de reutilizar software de forma rentable. En ese contexto, el enfoque específico no es simplicidad — es deuda técnica acumulada que eventualmente paraliza la operación.

El Modelo de Plataforma basado en un repositorio institucional es la inversión técnica necesaria para que la consultora pueda:

- Asumir más proyectos con el mismo equipo sin degradar la calidad.
- Reducir los costos de mantenimiento a medida que crece la cartera de clientes.
- Mitigar el riesgo de rotación de personal mediante un estándar documentado.
- Consolidar activos intelectuales reales que diferencien a la consultora en el mercado.